

Servo Joint Drive — How The Code Runs

A walk through the firmware: boot, one control period, the fault path, and what is alive at each bring-up stage.

STM32H743VIH6 · 16 kHz FOC · dual iC-MU200 · EtherCAT

1. The one idea everything else follows from

Every per-period activity that hardware can do, hardware does. TIM1 drives a slaved sequencer that pulses both encoder latch lines, starts both encoder reads by DMA, and triggers the converters — all without the CPU. Only when every input is resident does a single interrupt fire, and that interrupt is the FOC ISR.

Three consequences run through the whole codebase:

- **The ISR never waits.** There is no readiness check anywhere in `foc_isr()`, because the data existing is what caused the ISR to begin.
- **Every sample has a fixed relationship to every other.** The angle was latched at 10.94 μ s, the current sampled at 31.25 μ s, always.
- **The CPU keeps about two thirds of its capacity** for the fieldbus and the application.

One principle is qualified. Both timer break inputs are committed to Safe Torque Off and no current signal reaches a comparator, so there is **no sub-microsecond hardware overcurrent trip**. What replaces it is the oversampled analog watchdog path in §4 — roughly three times slower, and a declared limitation with a derating consequence rather than a solved problem.

2. Layers, and the rule that keeps them

Layer	Directory	Rate	Owns
L4 App	app/	main loop, 1 kHz	CiA 402 state machine, EtherCAT stack, object dictionary, clock discipline, supervisory checks
L3 Motion	motion/	4 kHz, pended	Position on the load encoder, velocity on the motor encoder, interpolation, feedforward, brake sequencing
L2 Torque	foc/	16 kHz, one ISR	Clarke, Park, current PI, SVPWM, angle advance, tier-2 checks, scope logging
L1 HAL	hal/	—	The only layer that knows pin numbers, converter channels, timer instances, scaling constants, frame formats
L0 Silicon	platform/	—	CubeMX generated. Regenerating touches nothing else.

The rule that gives this its value: **L2 and L3 contain no register access and no pin numbers**. Everything device-specific lives in L1, and the build enforces it two ways — CMake links `foc` and `motion` against a headers-only `hal_iface` target, so a HAL call is a link error; and `Tools/layer_check.py` catches what CMake cannot see, such as a register write through a hand-made pointer.

The payoff is that `foc/` and `motion/` compile and run on a PC against a plant model. If they do, the boundary is real. If they do not, the compiler errors are an exhaustive list of your layer violations.

Two files cross the boundary upward, deliberately

File	Calls upward	Why it must
<code>hal/isr_vectors.c</code>	<code>foc_isr()</code> , <code>motion_task()</code> , <code>supervisor_task()</code>	The vector body is register work — clearing JEOS, testing watchdog flags — and cannot live in L2.

File	Calls upward	Why it must
hal/hal_safety.c	foc_reset_integrators(), motion_reset_integrators(), scope_log_freeze()	§5.5 steps 3 and 6. A registered callback could be silently unregistered; a direct call cannot — it is a link error.

No third file may. The layer check whitelists exactly these two.

3. Boot: from reset to the first control period

Configuration and activation are split, and the split is what makes the ordering safe.

#	What runs	Where	Why here
1	Reset_Handler	startup_stm32h743xx.s	Copies .data and the ITCM text from flash, zeroes .bss. The ITCM copy is what makes ITCM_FUNC mean anything — without it the first call jumps into uninitialised RAM.
2	SystemClock_Config()	platform/Core/Src/main.c	HSE 24 MHz → PLL1P 480 MHz, AHB 240 MHz. Every timing constant in the project derives from the 240 MHz timer clock.
3	MX_*_Init() × 17	platform/Core/Src/main.c	Configures every peripheral and starts none of them. TIM1 gets ARR 7500 and its BDTR bits but CEN stays clear. This is why CubeMX's ordering is harmless.
4	servo_main()	app/servo_main.c	One line in USER CODE BEGIN 2. Everything after peripheral configuration is ours.

Inside servo_main()

Step	Call	What it does, and what breaks without it
1	hal_init_clocks()	MPU regions and caches. Must precede anything that places a buffer, because the non-cacheable attribute on D2 is what keeps DMA and the core agreeing.
2	hal_persist_valid()	Enables the backup SRAM clock, disables backup-domain write protection, checks the CRC. Forget the write protection and every later write is silently discarded .
3	hal_init_safety()	Brake engaged, both STO channels asserted, TIM1 outputs armed with MOE off. Does not fail when STO is asserted — at power-on it usually is, and refusing here would stop the drive ever reaching a state where it could report why.
4	hal_init_adc()	Calibrate, measure the three offsets with the bridge off, arm the analog watchdogs. Offsets before watchdogs, necessarily: the thresholds are relative to the amplifier's resting point.
5	hal_init_encoders()	Poll NERR up to 20 ms, write the iC-MBE registers, run INIT, read back the measured line delay. Slow, and it must finish before the sequencer starts requesting reads.
6	foc_init(), motion_init()	Control state zeroed, first-period flag set. Without that flag the very first ISR measures a meaningless interval and trips on a missed period before it has run once.
7	hal_init_fieldbus()	LAN9253 reset, 39-byte SETCFG, verify by reading the ID.
8	hal_init_sequencer())	TIM5, TIM6 armed-not-running, TIM3, TIM2, TIM4 — then TIM1 last . From the instant this returns, the system is live at 16 kHz.

Why TIM1 starts last, and why hal_pwm.c does not start it. The moment CEN is set, the carrier pulses both GETSENS lines, raises two DMA requests and triggers three converters — every 62.5 μs, forever. Every one of those consumers must already exist. The function that knows the order should be the one that calls it, so hal_init_sequencer() owns hal_pwm_carrier_start().

The slave-mode trap that costs a day. TIM2, TIM3 and TIM4 are in slave reset mode on ITR0. Reset mode zeroes CNT when the trigger arrives — it does **not** enable the counter. CEN must still be set on each slave independently. A slave with CEN clear sits at zero being repeatedly reset, produces nothing, and reports nothing.

4. One PWM period, in order

62.5 μ s. The CPU appears once, near the end. Times are from the carrier trough.

Time	Actor	What happens
0	TIM1 reload	Preloaded compare values take effect. The duty computed last period starts being applied now — this is the actuation delay the angle advance term compensates.
10.94 μ s	TIM1 → OC4REF	Counter passes CCR4 = 2625 on the up -count. TRGO rises. TIM2, TIM3 and TIM4 all reset to zero on that one edge.
10.94 μ s	TIM4 CH1+CH4	Both GETSENS pins go high together. Both iC-MBE masters start a BiSS frame. This instant is the measurement.
11.94 μ s	TIM4 CH1+CH4	GETSENS falls. 1 μ s pulse — long enough to register, and it must be low before EOT rises or the master raises nAGSERR.
≈14.3 μ s	iC-MBE ×2	Frames terminate. Each master verifies the CRC, writes the result into its own RAM, raises EOT and swaps banks. The MCU has done nothing.
16.94 μ s	TIM4 CH2+CH3	Two compare matches raise two DMA requests. Each writes one word into SPI→CR1, setting CSTART on an already-armed transfer.
≈19.6 μ s	SPI + DMA	Both read-outs complete. Status byte and position resident in D2 SRAM. 11.5 μs of margin before the converter needs them.
31.17 μ s	TIM1 → OC5REF	TRGO2 fires the injected groups. ADC1 and ADC2 sample phases B and C simultaneously; ADC3 takes phase A; ADC1 rank 2 takes bus voltage.
31.25 μ s	carrier peak	All three low-side devices conducting — the only instant the shunts carry phase current. The sampling aperture is centred here.
≈32.0 μ s	ADC1 JEOS	The one interrupt. ADC_IRQHandler → foc_isr().
≈36 μ s	ISR exit	New duty preloaded, feedback published, streams re-armed.
36 – 73.4 μ s	—	Idle on the acquisition path. 37 μ s of nothing.

Why the encoder chain sits where it does. The angle is latched at 10.94 μ s but the current is sampled at 31.25 μ s, so the angle is 20.3 μ s stale when it meets its currents. Latching later would reduce that — but the BiSS frame plus the read-out needs about 8.7 μ s, and the margin to the converter is what protects against a frame duration that has been computed rather than measured. CCR4 = 2625 spends staleness, which the advance term compensates, to buy margin, which nothing else can.

And why CCR4 must stay $\equiv 375 \pmod{750}$. TIM2 emits a protection trigger every 750 counts. At 2625 the samples land at 29.69 and 32.81 μ s, so the carrier peak sits exactly midway in a 3.125 μ s gap and the injected trigger has 1.5 μ s of clear air either side. Move CCR4 to any other residue and a protection conversion starts colliding with the control sample.

Running alongside, all period: the protection lane

TIM2 CH2 toggles every 750 counts. All three ADCs convert their phase on **both** edges, twenty times per period, 320 kHz. **Nothing reads the results.** Each converter's analog watchdog compares in hardware and stays silent unless a threshold is crossed.

That is the whole reason the rate is affordable. A CPU reading 320 kHz × 3 would be a million interrupts a second; a watchdog that produces nothing costs zero. Worst-case detection is one sample interval plus a conversion plus interrupt entry — about 3.5 μ s, against 62.5 μ s if the injected group were the only measurement.

Toggle mode gives **one edge per wrap**, so the ADC regular trigger must be set to both edges. Get it wrong and the rate silently halves to 160 kHz, doubling the detection latency that the R1 derating calculation assumes. Countable at stage 1: twenty conversions per period, not ten.

5. Inside foc_isr()

Thirteen steps. The order is the design: duty reaches the gates before anything that can wait.

#	Step	Note
1	hal_deadline_kick()	First. If anything below hangs, TIM6 expires and its priority-0 handler forces the safe state. Reloading later leaves a window in which a hang is undetectable.
2	TIM5 interval → classify	TIM1's counter is periodic: it catches a late entry but not a skipped period, which leaves it looking normal. A software counter cannot help either — it counts ISR executions, so it cannot notice its own absence. Missed ⇒ safe state, no limping.
3	One pointer copy each	params and current_ref taken once. Re-reading mid-ISR reintroduces the tear the double buffer exists to avoid.
4	hal_adc_read(), hal_enc_read() ×2	Four JDR registers and two polled DMA flags. No waiting anywhere.
5	Encoder validity	EOT, CRC verdict, nAGSERR, nDELAYERR — all from the status byte that arrives ahead of the position, so the checks cost nothing.
6	tier2_fast()	Current sum first, because everything after it assumes the currents are trustworthy.
7	angle_advance(), angle_sincos()	Sampled once, advance applied once, one sin/cos pair used for both transforms. That makes coherence structural rather than something to remember.
8	clarke_2ph → park → PI ×2 → inv_park → svpwm	Two currents carry all the information there is for a three-wire motor.
9	hal_pwm_set_duty()	Output. Everything below can wait; this cannot.
10	Velocity accumulation	Accumulate every period, publish every fourth. Four differences telescope to the total displacement, so nothing is discarded and the noise floor drops fourfold.
11	Publish feedback, pend tasks	The ISR never calls motion or the supervisor — it pends them and returns.
12	hal_enc_rearm()	Clear SPI EOT, clear DMA TC before re-arming, cycle PC13, fire the PDVALID reset.
13	scope_log_write()	Last, so isr_ticks measures the whole period's work.

Clear-then-arm, in step 12. Arm-then-clear leaves a window in which a fast transfer sets the TC flag and it is immediately cleared, so a genuinely missing frame reads as present next period. That is a silently wrong angle — the failure mode the whole acquisition design is arranged to avoid.

6. Interrupts: who preempts whom

Priority follows rate. Lower number wins; priority group 4 gives sixteen preemption levels and no subpriority.

Prio	Vector	Serves	Deadline
0	TIM1_BRK_IRQn	Break / STO trip. Hardware already acted; this records why.	immediate
0	ADC3_IRQn	Phase A watchdog, on its own vector — the R3 mitigation.	immediate
0	TIM6_DAC_IRQn	Deadline monitor. Must preempt the FOC ISR.	immediate
1	ADC_IRQn	FOC ISR, plus the phase B/C watchdogs that share it.	62.5 μ s
2	— reserved —	Held free for a future latency-critical need.	—
3	TIM3_IRQn	SYNC0 capture; starts the process-data cycle.	~100 μ s
4	TIM7_IRQn	Motion task, pended. TIM7 borrowed as a vector number only.	250 μ s
5	DMA2_Stream2/3	Fieldbus transfer complete; advances the state machine.	~100 μ s
6	LPTIM1_IRQn	Supervisory task, pended.	1 ms
10+	EXTI, SPI, USART	LAN9253 events, STO monitors, 485 error flags.	none
—	main loop	Whatever remains, which is most of it.	none

Priority 0 for the deadline monitor is not a preference. To catch a control ISR stuck in a loop it must preempt that ISR, which sits at priority 1. Equal priority means no preemption — so ADC_IRQn at 0 alongside TIM6 would make the monitor useless in exactly the case it exists for.

Worst case is the one-in-sixteen coincidence period, when the control ISR, the motion task, the supervisory task and the fieldbus sync all land together: roughly 33 μ s of 62.5, about 53% utilisation. Typical periods are nearer 10%.

Low priority does not mean starved. A pending interrupt stays pending — the controller latches the bit and services it as soon as the CPU drops below its level. The supervisory task is deferred by at most ~22 μ s against a 1 ms period.

The two things that will actually hurt you, and neither is interrupt count. First, **critical sections**: any interrupt disable delays the control ISR by its full duration regardless of priority, and delays the fast protection path with it. Vendor fieldbus stacks contain long ones — audit every one. Second, **vendor interrupt dispatch**: routing the control ISR through HAL_ADC_IRQHandler makes it walk every flag and evaluate callbacks first, hundreds of cycles of *variable* overhead, which then shows up in the §15 duration histogram as spread that looks like a memory placement fault.

7. What happens when something goes wrong

Four tiers, ordered by response time. Every one converges on the same terminal state.

Tier	Detects	Latency	Path
T0	Desaturation, shoot-through interlock, driver UVLO	<1 μ s	Autonomous inside the gate driver. With the comparator tier absent this is the only sub-microsecond protection in the system.
T1'	Phase overcurrent, any of three phases	$\approx$ 3.5 μ s	Analog watchdog $\rightarrow$ ADC_IRQHandler or ADC3_IRQHandler $\rightarrow$ hal_safe_state()
T2	Current sum, filtered overcurrent, bus, encoder status, missed period	62.5 μ s	tier2_fast() inside foc_isr() $\rightarrow$ hal_safe_state()
T3	Temperature, both cross-checks, fieldbus watchdog, over-speed, I ² t	1 ms – s	supervisor_task() $\rightarrow$ hal_safe_state()
L3	The ISR stopped entirely	100 μ s	TIM6 $\rightarrow$ HAL_TIM_PeriodElapsedCallback() $\rightarrow$ hal_safe_state()
L4	The whole chain dead	250 ms	IWDG $\rightarrow$ processor reset. Refreshed only by the supervisory task, which only runs if the ISR pended it — so the refresh proves the chain end to end.

hal_safe_state(), step by step

#	Action	Why in this position
1	hal_pwm_break_now()	First, before anything that takes time — until this runs the bridge is still driving the last duty. EGR.BG latches exactly as a hardware break would, so recovery needs the same explicit action.
2	sto_assert_both()	The independent path, not dependent on the timer.
3	foc_reset_integrators() motion_reset_integrators())	The step people leave out. Skip it and re-enable trips instantly, because the current loop restarts with a wound-up integrator demanding full voltage into a stationary rotor. It presents as a hardware fault and is not one.
4	hal_brake_engage()	Commanded, not waited for . Torque already went in steps 1 and 2, and blocking tens of milliseconds inside a priority-0 handler is not an option. The load is unheld for the brake's engagement time — inherent to STO, and it belongs in the risk register.
5	fault_set(cause)	Atomic bit-set only. Runs at several priorities, so a lower-priority writer must not lose a bit set by a higher one.
6	scope_log_freeze()	Idempotent. One real fault usually cascades — overcurrent trips the watchdog, the safe state removes torque, the current collapse trips a tier-2 check. If each froze the buffer the last would win and you would capture the consequences, not the cause.
7	hal_deadline_stop()	Otherwise TIM6 expires 100 μ s later and trips again, burying the real cause under a deadline fault.

Deliberately absent: any retry, timeout or automatic recovery. Re-enable happens only by explicit command through the drive state machine, and the fault-reset bit is edge-triggered so a master that leaves it set cannot create a retry loop.

Recovery, in drive_profile.c


```
case ST_FAULT:
    if (s.fault_reset_edge && sto_ok) {
        hal_pwm_clear_break_flags(); /* BEFORE the fault word */
        fault_clear_all();
        foc_reset_integrators();
        motion_reset_integrators();
        s.state = ST_SWITCH_ON_DISABLED;
    }
```

The order matters: `hal_pwm_outputs_enable()` refuses while a break flag is latched, so clearing the fault word first would advertise a drive ready to enable that then silently refuses.

8. Bring-up, stage by stage

Stages 3 through 8 are **one binary**. The mode is a runtime field in `params_t`, not a set of `#ifdefs` — because §15 wants worst-case ISR timing measured with all instrumentation enabled, and if each stage were a different build then every number recorded during commissioning would belong to a different program.

The one thing that stays compile-time is the ceiling on danger: below stage 3, `hal_pwm_outputs_enable()` returns false unconditionally. A runtime flag can be set wrong from the object dictionary; that cannot.

St	Scope	Files	Pass criterion
0	Protection. No motor.	<code>hal_safety</code> , <code>hal_adc_watchdogs</code> , <code>isr_vectors_fault_paths</code>	Gate driver protection verified from the datasheet. Watchdog limits set below normal current; an injected fault reaches the safe state within the measured latency, on each phase in turn . STO verified from both channels and from software assert. Nothing else begins until this passes.
1	Clocks, carrier, every trigger, converters. Motor disconnected, fixed duty.	<code>hal_pwm</code> , <code>hal_sequencer</code> , <code>hal_time</code> , <code>scope_log</code>	Every trigger instant scope-verified against the carrier and confirmed never to collide with protection sampling. Offsets converge. Bus voltage reads true. Dead time correct. One ISR pulse per period — two means an extra source is enabled. Also sets <code>CARRIER_ENTRY_TOL</code> and <code>LATE_ENTRY_LIMIT</code> from the jitter histogram.
2	Encoders, synchronous acquisition, no motion.	<code>hal_encoder</code> , <code>hal_encoder_ll</code>	Both angles correct and repeatable by hand. Latch instants indistinguishable on a scope. Gear ratio between the two verified. Both frames complete before the converters, with margin. The polled TC flag never false. The one capture worth saving: <code>GETSENS</code> , <code>EOT</code> and chip-select on one screen. It fixes <code>CCR2/CCR3</code> from the measured line delay.
3	Open loop, forced angle. Motor connected.	<code>transforms</code> , <code>angle</code> , <code>svpwm</code>	Smooth rotation in the commanded direction. Currents sinusoidal and balanced. Sign of every sensed current confirmed . Sum check quiet. First stage where <code>CTRL_MODE_OPEN_LOOP_VOLTAGE</code> and <code>angle_forced_step()</code> are used — no commutation offset needed yet.
4	Current loop closed on the forced angle.	<code>current_pi</code> , <code>tier2_checks</code>	Commanded current tracked with no oscillation, well-damped step response, no windup on saturation. Where the scope buffer earns its keep: a current step settles in ~30 periods, which no watch variable can sample and no external scope can see.
5	Commutation offset calibration, then closed-loop FOC.	— (calibration procedure)	Torque per amp at expectation. Angle advance tuned by minimising current at constant load. No degradation as speed rises — the test that proves the sampling is coherent. Writes <code>COMMUTATION_OFFSET</code> back into <code>motor_params.h</code> .
6	Velocity, then position on the load encoder. Torque sensor integrated.	<code>motion/*</code> , <code>supervisor</code>	Disturbance rejection acceptable. No resonance excitation at the joint. Both cross-check thresholds characterised by measurement, not guessed . If resonance does appear, §8 says that is when to address it — not before.
7	Fieldbus: state machine, process data, sync capture, clock discipline, interpolation.	<code>hal_fieldbus</code> , <code>clock_discipline</code> , <code>fieldbus_app</code>	Phase loop locks and holds; error bounded across a thermal cycle. Watchdog and quick stop verified by pulling the cable with the joint under load .

St	Scope	Files	Pass criterion
8	Full system, production build.	hal_persist, fw_update, object_dict	Worst-case ISR duration measured with all instrumentation enabled. Overrun counter zero across an extended run. Every tier fault-injected. Scope buffer captures a real trip. Lock level 2 on TIM1's BDTR goes in here, and not before.

Protection is stage 0, not a later addition. A drive that cannot arm its protection tier must not turn a motor, which is why `build_config.h` refuses to compile a motor-connected stage while the watchdog limits are unset.

9. Where data lives, and who writes it

Each shared datum has exactly one writing context. Coherent multi-word sets cross domains by double buffer with a single-word swap, never field by field.

Datum	Writer	Readers	Mechanism
Encoder samples	DMA	Torque	Single buffer + shadow, TC flag polled, timestamped
Current reference	Motion	Torque	Single aligned word — atomic against interrupts
Feedback + status	Torque	Motion, Supervisor, Fieldbus	Double buffer, published at ISR exit
Velocity accumulator	Torque	Motion	Double buffer; ISR owns both slots so P4 holds
Position command	Fieldbus sync	Motion	Double buffer + interpolator state
Torque reading	USART3 DMA	Motion, Supervisor	Double buffer + validity flag + age stamp
Gains, parameters	Supervisor	Torque, Motion	Double buffer + swap flag, never field by field
Fault word	All tiers	Supervisor, Fieldbus	Atomic bit-set only; cleared solely by explicit reset

`contracts.c` carries a static assertion that `current_ref_t` is exactly one word and word-aligned. If it ever grows past 32 bits the atomic publish silently becomes a tearing multi-word write, and the assertion is the only thing that catches it.

Memory placement

Region	Size	Contents	The hazard it avoids
Instruction TCM	64 K	Control ISR, transforms, controllers, modulator, fault handlers	Zero wait state, never cached — §15 wants the ISR duration histogram spread near zero, and visible spread means something still runs from flash.
Data TCM	128 K	Stack, trig table, control state, integrators, decoded encoder values	No DMA reach at all. A DMA buffer placed here fails quietly: no error, no transfer.
D2 SRAM	288 K	Every DMA buffer, non-cacheable	Same domain as the controllers, so no bridge delay; non-cacheable removes stale-read bugs that appear only under load.
AXI SRAM	512 K	Scope buffer, firmware staging, object dictionary	The default for anything not named above.
Backup SRAM	4 K	Fault history, crash dump, calibration validity	Never zeroed at startup — it sits outside every zero-fill loop, which is what makes the guarantee hold.

Both hazards are solved by policy rather than vigilance: a non-cacheable region for every DMA buffer, and tightly-coupled memory for everything the core owns alone. That is set up once. The alternative — cache maintenance at every transfer site — must be got right by every future call site forever.

`Tools/section_check.py` verifies placement in the linker map, because section attributes are silently ignored more often than you would expect.

10. Reading a fault after the event

An encoder interface occupies the trace pins, so there is **no SWO and no ETM**. That removes every conventional way of watching a transient, and it is why the in-RAM scope is not a convenience but the primary instrument.

Approach	Why it does not work here
printf over UART	Formatting one line costs longer than the 62.5 μ s deadline, so the act of printing destroys the timing being printed about.
Halting in the debugger	The motor does not halt with the CPU. The bridge holds the last duty written and the loop regulating it is gone.
Live watch variables	The debugger polls at maybe 10 Hz; a current step settles in a millisecond.
An external oscilloscope	Cannot see <code>i_q</code> , an integrator, or the angle advance term. Those are not on any pin.

So the buffer runs circularly at all times and a fault freezes it. Eight channels at 16 kHz for half a second, 250 K in AXI SRAM, eight stores per period — about 0.05% of the budget.

The property that matters is pre-trigger capture. Logging tells you what happened after you decided to look. This tells you what happened before you knew to — including the 3 a.m. trip during a soak run with nobody watching.

Stage	Readout path	Effort
4	<code>gdb dump + Tools/scope_plot.py</code>	Already written. Channel names are parsed out of <code>scope_log.h</code> at run time, so they cannot drift from the struct.
6	UART command, CSV out	~50 lines in the main loop. Works without a debugger — matters for a soak run or a drive already in a machine.
7	<code>app/scope_readout.c</code> over CoE	The production path. A technician pulls a capture with TwinCAT, no debugger, no serial cable.

Alongside it, `hal_persist.c` keeps the fault history in backup SRAM. The scenario is a drive that trips in a machine, an operator who power-cycles it, and an engineer who arrives an hour later: the history says *which* trip, the scope buffer says *what led to it*.

11. What is still open

Nine measurements and one hardware question. None are firmware problems; all of them gate a bring-up stage.

#	Item	Gates	Who
1	MPC per encoder — the pole-pair count on each magnetic ring	PDLEN, TSIZE, over-speed trip, cross-check threshold	read the part number
2	Gate driver part number (dependency D1)	Dead time, CHxN polarity, and how severe R1 actually is	datasheet
3	INA241 propagation delay	T1' latency, therefore the R1 derating	datasheet
4	INA241 settling time	The duty clamp CCR_MAX	stage 3, scope
5	Shunt value × INA241 gain	Every current reading in mA	schematic
6	Bus divider ratio and impedance	v_bus_mv, and whether Vbus stays on ADC1 rank 2	schematic
7	Measured line delay per encoder	TIM4 CCR2/CCR3 final values	stage 2, the chip measures it
8	Board pull-downs on the gate driver inputs (D3)	The whole watchdog layer is worthless without them	schematic
9	STO assert response time (D4)	Which path is primary	measure
10	VBAT backed by a battery or supercap?	If not, backup SRAM survives reset but not a power cycle — and hal_persist.c becomes decoration	schematic

R1 is the only item that cannot be resolved in firmware. The fastest trip is around 3.5 μs instead of under one, which requires power-stage derating supported by a let-through energy calculation at the *measured* latency. That is a power-electronics calculation and it belongs to that discipline. The correct response is a derating decision made deliberately and recorded — not a firmware measure assumed to be equivalent to the hardware trip it replaces.

12. Deliberate deviations from the architecture

Recorded here so they are decisions rather than drift.

Deviation	Reason
common/ directory added (§13 tree has none)	Fixed-point types, data contracts and fault codes are needed by both L1 and L2 and depend on neither.
Multiturn dropped from hal_persist	Joint travel is under one revolution of the load encoder, so that encoder is absolute across the whole range. Fields kept as reserved rather than removed.
System identification and the IIR cascade removed	Costs: current-loop gains become a search rather than a calculation, §8's two-inertia question stays open, and both §8.3 thresholds stay guessed where §15 asks for them measured.
Brake is power-to-release, not power-to-engage	§3.1's wording inverts: 0% duty is engaged . Engaging is the slow direction — cutting duty only starts the coil current decaying.
hal_safety.c calls upward into L2 and L3	§5.5 steps 3 and 6 need it, and a registered callback can be silently unregistered on a fault path where silence is the worst outcome.

Deviation	Reason
Linker script lives outside platform/	CubeMX regenerates platform/STM32H743xx_FLASH.ld and would silently clobber every custom section — and the failure would be DMA buffers landing in DTCM, with no error.

The one sentence to keep

Hardware triggers, software computes. Every check that can be structural is structural — a link error, a static assertion, a build check — because discipline that exists only in a document decays within months, and the failures this design guards against are the ones that produce no error at all.